

# PARORA Hosting Guide for Meharry SACS

Sep 25, 2026 · @Methun

PARORA (Protein Agentic Rendering & Observation for Residue Analysis) is a browser app for exploring protein 3D structures by typing plain-English requests. This guide covers how to publish it at <https://meharry.edu/academics/schools/school-of-applied-computational-sciences/parora> on a server in our building, for a limited number of users.

## 1. What PARORA does

A user types a request such as "Show me hemoglobin and highlight the heme groups as ball and stick". A language model runs on the server itself, not in the cloud. It decides which built-in tools to call, fetches the structure, and draws it in an interactive 3D viewer (NGL.js, WebGL) in the browser.

Main capabilities of the full app ( `app.py` , 56 tools):

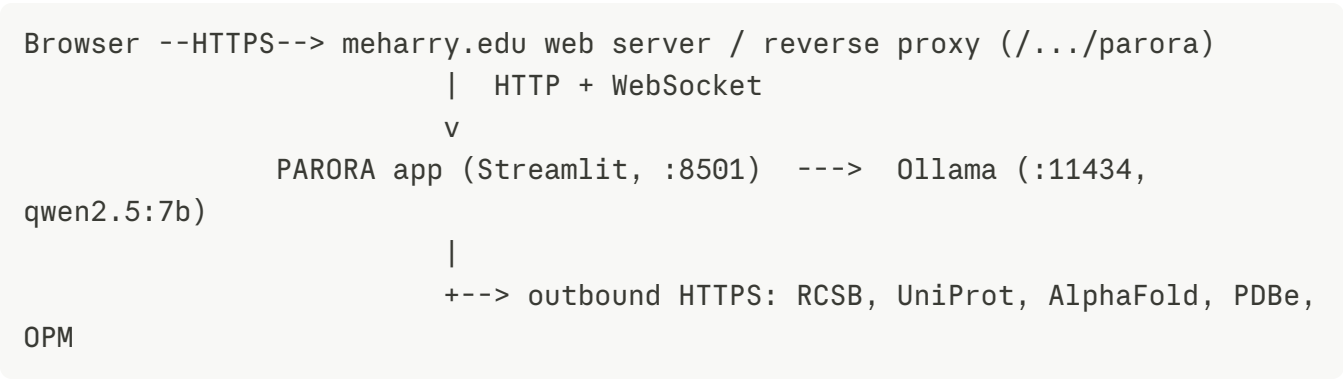
- **Find and load structures:** searches the RCSB Protein Data Bank and UniProt. Falls back to AlphaFold predicted models when no experimental structure exists. Users can also upload their own PDB files.
- **Visualize:** cartoon, surface, ball-and-stick and other styles, colors, labels, selections by chain, residue or ligand.
- **Analyze:** B-factor filtering, proximity queries, RMSD superposition, interaction detection (salt bridges, hydrogen bonds, disulfides, pi-stacking, metal sites), distance, angle and dihedral measurements, and structure-composition reports.
- **Prepare and simulate** (optional add-ons): structure cleanup, membrane building, Amber, GROMACS and Rosetta input files, QM and QM/MM (Gaussian ONIOM) inputs, ray-traced images, and fold-similarity search.

The app asks the user a clarifying question when a request is ambiguous, instead of guessing.

**Privacy point for IT:** no cloud AI service or API key is used. User prompts never leave our server. The server does make outbound calls to public science databases (RCSB, UniProt, AlphaFold, PDBe, OPM) to fetch structures.

## 2. How it is built

PARORA runs as two processes on one server: a Python web app (Streamlit, port 8501) and a local model server (Ollama, port 11434). A reverse proxy in front publishes it under the Meharry URL.



| Component        | What it is                                                      | Port  | Exposed publicly?      |
|------------------|-----------------------------------------------------------------|-------|------------------------|
| PARORA app       | protein-viz-agent/app.py , Python 3.12, Streamlit               | 8501  | Only through the proxy |
| Ollama           | Local LLM runtime serving qwen2.5:7b                            | 11434 | Never                  |
| Reverse proxy    | nginx or Apache on the host, or the main meharry.edu web server | 443   | Yes                    |
| Optional add-ons | AmberTools, PyMOL, DSSP, Foldseek (separate conda environments) | none  | No                     |

The repository has two other entry points, `server.py` (FastAPI) and `app_lite.py`. They have only 3 tools and are meant for development. **Host `app.py`**. It is also what the provided Docker image runs.

Important behaviors for hosting:

- **Streamlit uses WebSockets.** The proxy must pass WebSocket upgrades, or the page loads but never responds.
- **Each browser tab is its own session.** Chat, loaded structures and the viewer are kept per user in server memory. A restart clears all sessions.
- **One model serves everyone.** Ollama answers a few requests at once and queues the rest, so response time grows with active users.
- **There is no built-in login.** Access control must come from the proxy (Section 7).

- **Shared folders:** downloaded and uploaded files go to `structures/` , `prepared/` , `membranes/` and `simulations/` , shared by all users. Two users uploading files with the same name overwrite each other.

### 3. Server requirements

Recommended: a Linux server (Ubuntu 22.04 or 24.04 LTS) with an NVIDIA GPU of 12 GB+ VRAM, 32 GB RAM and 100 GB free SSD. It can run CPU-only, but each answer then takes tens of seconds and supports only 2 to 3 concurrent users.

| Resource | Minimum (CPU only, ~3 users)            | Recommended (GPU, ~10-20 users)                                         |
|----------|-----------------------------------------|-------------------------------------------------------------------------|
| OS       | 64-bit Linux, Ubuntu 22.04+             | Ubuntu 22.04 / 24.04 LTS                                                |
| CPU      | 8 cores, x86-64 with AVX2               | 16+ cores                                                               |
| RAM      | 16 GB                                   | 32-64 GB                                                                |
| GPU      | None                                    | NVIDIA, 12-24 GB VRAM (e.g. RTX 4070 Ti / A4000 or better), driver 535+ |
| Disk     | 40 GB free SSD                          | 100+ GB free SSD                                                        |
| Network  | Outbound HTTPS to the science databases | Same, plus inbound 443 via proxy only                                   |

Disk use breakdown (approximate):

| Item                                                                                              | Size                                | Required?               |
|---------------------------------------------------------------------------------------------------|-------------------------------------|-------------------------|
| Ollama model <code>qwen2.5:7b</code>                                                              | ~4.7 GB                             | Yes                     |
| PARORA Docker image or conda env (Python, Streamlit, MDAnalysis, RDKit)                           | ~3 GB                               | Yes                     |
| Docker / Ollama / OS overhead                                                                     | ~5 GB                               | Yes                     |
| User data ( <code>structures/</code> , <code>prepared/</code> , <code>simulations/</code> , logs) | grows ~1-5 GB per year of light use | Yes, clean periodically |
| AmberTools conda env                                                                              | ~3-5 GB                             | Optional                |
| Foldseek PDB database                                                                             | ~4.2 GB                             | Optional                |

| Item                   | Size    | Required? |
|------------------------|---------|-----------|
| PyMOL, DSSP conda envs | ~1-2 GB | Optional  |

Memory notes: the model needs about 6 GB of RAM or VRAM, plus about 1-2 GB per request served in parallel (context window is 16,384 tokens). The web app uses about 300-600 MB per active user.

If no Linux server is available, a Mac with Apple Silicon and 32 GB RAM also works well. Ollama uses the Apple GPU natively there.

## 4. Software to install

Use the Docker path (Option A) unless the optional simulation add-ons are needed. Docker keeps Python dependencies isolated from the rest of the server. The conda path (Option B) is the only one that includes AmberTools and PyMOL.

| Software                     | Version           | Needed for                                             | Install source                                                                                                               |
|------------------------------|-------------------|--------------------------------------------------------|------------------------------------------------------------------------------------------------------------------------------|
| Git                          | any recent        | Getting the code                                       | OS package manager                                                                                                           |
| Ollama                       | latest            | Runs the language model                                | <a href="https://ollama.com/download">ollama.com/download</a> ( <code>curl -fsSL https://ollama.com/install.sh   sh</code> ) |
| Model<br>qwen2.5:7b          | pulled via Ollama | The AI agent                                           | <code>ollama pull qwen2.5:7b</code>                                                                                          |
| NVIDIA driver + CUDA runtime | driver 535+       | GPU speed (optional but recommended)                   | NVIDIA / Ubuntu <code>ubuntu-drivers</code>                                                                                  |
| Docker Engine                | 24+               | Option A: runs PARORA container                        | <a href="https://docs.docker.com/engine/install">docs.docker.com/engine/install</a>                                          |
| Miniconda or Miniforge       | latest            | Option B: Python 3.12 env from <code>parora.yml</code> | <a href="https://docs.conda.io">docs.conda.io</a>                                                                            |
| nginx (or Apache httpd)      | any recent        | Reverse proxy, HTTPS, login, user cap                  | OS package manager                                                                                                           |

| Software        | Version | Needed for | Install source                                      |
|-----------------|---------|------------|-----------------------------------------------------|
| TLS certificate | -       | HTTPS      | University certificate or Let's Encrypt ( certbot ) |

Python packages come from `protein-viz-agent/requirements.txt` (Streamlit, FastAPI, Ollama client, MDAnalysis, NumPy, pandas, Biopython, RDKit, rcsb-api, requests, PyYAML). Docker and `run.sh` install them automatically.

Optional scientific add-ons (Option B only). Each lives in its own conda environment. Without them the app still runs; those specific tools report "unavailable". The repo script `bash setup_tools.sh` walks through installing them.

| Add-on                  | Enables                                             | Install                                                                                                                                                                  |
|-------------------------|-----------------------------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| AmberTools              | Structure prep with hydrogens, membranes, MD, QM/MM | <code>conda create -n ambertools -c conda-forge ambertools</code>                                                                                                        |
| PyMOL (open source)     | Ray-traced publication images                       | <code>conda create -n pymol-render -c conda-forge pymol-open-source</code>                                                                                               |
| DSSP                    | Computed secondary-structure topology               | <code>conda create -n dssp -c conda-forge dssp</code>                                                                                                                    |
| Foldseek + PDB database | Structural similarity search                        | <code>conda create -n foldseek -c conda-forge -c bioconda foldseek</code> , then <code>foldseek databases PDB protein-viz-agent/foldseek_db/pdb /tmp/fs (~4.2 GB)</code> |

Environment settings the app reads (all optional; defaults in `protein-viz-agent/config.yaml`):

| Variable                 | Purpose                    | Value for this deployment                                                                              |
|--------------------------|----------------------------|--------------------------------------------------------------------------------------------------------|
| <code>OLLAMA_HOST</code> | Where the app finds Ollama | <code>http://localhost:11434</code> (conda) or <code>http://host.docker.internal:11434</code> (Docker) |

| Variable                                                                                                                                                                    | Purpose                            | Value for this deployment                             |
|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------|------------------------------------|-------------------------------------------------------|
| <code>PARORA_MODEL_APP</code>                                                                                                                                               | Override the model                 | leave unset ( <code>qwen2.5:7b</code> )               |
| <code>PARORA_NUM_CTX</code>                                                                                                                                                 | Model context window               | leave at 16384                                        |
| <code>PARORA_LOG_DIR</code> ,<br><code>PARORA_LOG_LEVEL</code>                                                                                                              | Log location and verbosity         | e.g. <code>/var/log/parora</code> , <code>INFO</code> |
| <code>OLLAMA_NUM_PARALLEL</code><br>(Ollama side)                                                                                                                           | Requests the model answers at once | 2-4 depending on GPU memory                           |
| <code>AMBERHOME</code> ,<br><code>PACKMOL_MEMGEN</code> ,<br><code>PYMOL_PYTHON</code> , <code>DSSP_BIN</code> ,<br><code>FOLDSEEK_BIN</code> ,<br><code>FOLDSEEK_DB</code> | Optional add-on locations          | set by <code>run.sh</code> automatically              |

## 5. Installation steps

All commands assume Ubuntu, a service account named `parora` , and the code in `/opt/parora` . Adjust paths as needed. The app must listen only on `127.0.0.1` ; the public reaches it only through the proxy.

### 5.1 Common steps

1. Create a service user and get the code. The repository is

`https://github.com/xperthut/PARORA` ; ask the project owner for read access if it is private.

```
sudo useradd -r -m -d /opt/parora -s /bin/bash parora
sudo -u parora git clone https://github.com/xperthut/PARORA.git
/opt/parora/PARORA
```

2. (GPU servers) Install the NVIDIA driver, reboot, and confirm with `nvidia-smi` .
3. Install Ollama. The installer creates a `systemd` service bound to `127.0.0.1:11434` .

```
curl -fsSL https://ollama.com/install.sh | sh
ollama pull qwen2.5:7b
```

4. Tune Ollama for shared use: `sudo systemctl edit ollama`, add the lines below, then `sudo systemctl restart ollama`.

```
[Service]
Environment="OLLAMA_NUM_PARALLEL=2"
Environment="OLLAMA_KEEP_ALIVE=-1"
Environment="OLLAMA_MAX_QUEUE=64"
```

`KEEP_ALIVE=-1` keeps the model loaded so the first user of the day does not wait.  
Raise `NUM_PARALLEL` to 4 on a 24 GB GPU.

5. Create a Streamlit config at `/opt/parora/PARORA/protein-viz-agent/.streamlit/config.toml`. The `baseUrlPath` makes the app work under the Meharry sub-path (Section 6).

```
[server]
address = "127.0.0.1"
port = 8501
headless = true
baseUrlPath = "academics/schools/school-of-applied-computational-
sciences/parora"
maxUploadSize = 50          # MB per uploaded structure file
enableCORS = true
enableXsrfProtection = true

[browser]
gatherUsageStats = false    # no telemetry to Streamlit
```

## 5.2 Option A: Docker (recommended)

1. Install Docker Engine and add the `parora` user to the `docker` group.
2. Build the image from the repo root, including the Streamlit config:

```
cd /opt/parora/PARORA
docker build -t parora -f protein-viz-agent/Dockerfile .
```

The Dockerfile does not copy `.streamlit/`, so mount it at run time (next step).

3. Run the container with host networking, so it reaches Ollama on `127.0.0.1` and binds only to localhost:

```
cd /opt/parora/PARORA/protein-viz-agent
mkdir -p structures membranes prepared simulations logs
docker run -d --name parora --restart unless-stopped --network host \
  -e OLLAMA_HOST=http://127.0.0.1:11434 \
  -v "$PWD/.streamlit:/app/.streamlit:ro" \
  -v "$PWD/structures:/app/structures" \
  -v "$PWD/membranes:/app/membranes" \
  -v "$PWD/prepared:/app/prepared" \
  -v "$PWD/simulations:/app/simulations" \
  -v "$PWD/logs:/app/logs" \
  parora streamlit run app.py --server.address=127.0.0.1
```

Do not use the repo's `deploy.sh` on this server. It is written for a Mac laptop: it publishes port 8501 on all interfaces and relies on `host.docker.internal`, which does not exist on Linux by default.

#### 4. Test locally on the server: `curl -I`

`http://127.0.0.1:8501/academics/schools/school-of-applied-computational-sciences/parora/` should return `200`.

### 5.3 Option B: conda (includes simulation add-ons)

1. As the `parora` user, install Miniforge into `~/miniforge3`.
2. Optionally run `bash setup_tools.sh` for AmberTools, PyMOL, DSSP and Foldseek.
3. Start once by hand to verify: `bash run.sh`. It builds the `parora` env from `parora.yml`, installs `requirements.txt`, checks Ollama, finds add-ons, and starts the app.
4. Make it a service with `/etc/systemd/system/parora.service`:

```
[Unit]
Description=PARORA protein viewer
After=network-online.target ollama.service
Requires=ollama.service

[Service]
User=parora
WorkingDirectory=/opt/parora/PARORA
Environment=PARORA_LOG_DIR=/opt/parora/PARORA/protein-viz-agent/logs
ExecStart=/bin/bash /opt/parora/PARORA/run.sh
Restart=on-failure
RestartSec=5

[Install]
WantedBy=multi-user.target
```

5. `sudo systemctl daemon-reload && sudo systemctl enable --now parora.run.sh` starts the app from `protein-viz-agent/`, so it picks up the `.streamlit/config.toml` from step 5.1.5.

## 6. Publishing at the Meharry URL

Target address: `https://meharry.edu/academics/schools/school-of-applied-computational-sciences/parora/`. The web server that answers for `meharry.edu` forwards everything under `/parora/` to the PARORA app. Streamlit's `baseUrlPath` (step 5.1.5) matches this path exactly, so the proxy passes the path through unchanged.

Pick the case that fits the current setup:

| Case            | When                                                                                                          | What to do                                                            |
|-----------------|---------------------------------------------------------------------------------------------------------------|-----------------------------------------------------------------------|
| A. Same machine | PARORA runs on the server that hosts meharrry.edu, or on a box whose nginx/Apache already serves meharrry.edu | Add the proxy block below; app listens on <code>127.0.0.1:8501</code> |

| Case                                       | When                                                                       | What to do                                                                                                                                                                                                                |
|--------------------------------------------|----------------------------------------------------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| B. Separate PARORA box                     | meharry.edu web server is a different machine in the building              | Add the proxy block on the meharry.edu server pointing to the PARORA box's internal IP. On the PARORA box, set <code>address</code> to its internal IP and firewall port 8501 to accept only the web server's IP          |
| C. Main site on a CMS we cannot proxy from | meharry.edu is hosted by a CMS vendor or does not allow custom proxy rules | Publish at a subdomain such as <code>parora.meharry.edu</code> (DNS A record to the PARORA box), set <code>baseUrlPath = ""</code> , and make the <code>/parora</code> page on the main site a link or 302 redirect to it |

## 6.1 nginx proxy block

Place inside the existing `server { listen 443 ssl; server_name meharry.edu; ... }` block. Replace `127.0.0.1` with the PARORA box's internal IP in Case B.

```
# once, in the http {} context
map $http_upgrade $connection_upgrade { default upgrade; '' close; }

# inside the meharry.edu server {} block
location = /academics/schools/school-of-applied-computational-sciences/parora
{
    return 301 $uri;
}
location /academics/schools/school-of-applied-computational-sciences/parora/
{
    proxy_pass http://127.0.0.1:8501;          # no trailing slash: path
passes through
    proxy_http_version 1.1;
    proxy_set_header Upgrade $http_upgrade;    # WebSocket, required by
Streamlit
    proxy_set_header Connection $connection_upgrade;
    proxy_set_header Host $host;
    proxy_set_header X-Real-IP $remote_addr;
    proxy_set_header X-Forwarded-For $proxy_add_x_forwarded_for;
    proxy_set_header X-Forwarded-Proto $scheme;
```

```

    proxy_read_timeout 86400;                # keep long-lived sessions
open
    proxy_buffering off;
    client_max_body_size 50m;                # matches maxUploadSize
}

```

## 6.2 Apache equivalent

Enable modules: `sudo a2enmod proxy proxy_http proxy_wstunnel rewrite headers`.

```

<Location /academics/schools/school-of-applied-computational-
sciences/parora/>
    ProxyPass          http://127.0.0.1:8501/academics/schools/school-of-
applied-computational-sciences/parora/ upgrade=websocket
    ProxyPassReverse   http://127.0.0.1:8501/academics/schools/school-of-
applied-computational-sciences/parora/
    ProxyPreserveHost  On
    RequestHeader      set X-Forwarded-Proto "https"
</Location>

```

`upgrade=websocket` needs Apache 2.4.47+. On older Apache, add a `RewriteRule` to `ws://` for the `_stcore/stream` path.

## 6.3 HTTPS and verification

- Reuse the existing meharry.edu certificate in Cases A and B. Case C needs a certificate for the subdomain (university CA or `certbot --nginx -d parora.meharry.edu`).
- Reload: `sudo nginx -t && sudo systemctl reload nginx` (or `apachectl configtest && systemctl reload apache2`).
- Open the URL in a browser. The PARORA banner and chat box should appear, and typing "show 1CRN" should draw a small protein within about 30 seconds.
- If the page shows but spins forever, WebSocket upgrade is not getting through (see Section 9).
- Firewall: only 443 (and 80 for redirects) open to the internet. Ports 8501 and 11434 must never be reachable from outside.

## 7. Limiting use to a set number of people

PARORA has no login of its own, so all limits are enforced at the proxy. Use two layers together: accounts decide **who** may use it, and a connection cap decides **how many at once**. Both work without changing PARORA's code.

| Layer                     | Controls                           | Tool                                                | Suggested value                 |
|---------------------------|------------------------------------|-----------------------------------------------------|---------------------------------|
| 1. Accounts               | Who can get in (N named people)    | nginx Basic Auth, or campus SSO                     | one account per approved person |
| 2. Concurrent-session cap | How many use it at the same moment | nginx <code>limit_conn</code> on the WebSocket path | 10 on a GPU server, 3 CPU-only  |
| 3. Per-person cap         | Tabs one person can hold open      | <code>limit_conn</code> keyed on username           | 2                               |
| 4. Request rate           | Abuse or scripted hammering        | nginx <code>limit_req</code>                        | 30 requests/min per user        |
| 5. Network (optional)     | Campus or VPN only                 | <code>allow</code> / <code>deny</code> by IP range  | Meharry ranges                  |

### 7.1 Named accounts (Basic Auth)

Simplest option, good for up to about 50 people.

```
sudo apt install apache2-utils
sudo htpasswd -c /etc/nginx/parora.htpasswd firstuser # -c only for the
first user
sudo htpasswd /etc/nginx/parora.htpasswd seconduser # add each approved
person
sudo htpasswd -D /etc/nginx/parora.htpasswd olduser # remove access
```

The number of lines in `parora.htpasswd` is the number of people allowed. Changes take effect immediately, with no reload.

For a larger or longer-running audience, use the university's single sign-on instead: put [oauth2-proxy](#) in front (works with Microsoft Entra ID / Google Workspace) with `auth_request`, or Shibboleth/SAML on Apache, restricted to an approved group. The group's membership then defines the N people.

## 7.2 nginx configuration with all caps

Replace the Section 6.1 location block with this version. The WebSocket at

`.../_stcore/stream` is held open for as long as a browser tab is open, so counting those connections counts active sessions.

```
# http {} context
limit_conn_zone $server_name zone=parora_total:1m;      # all users combined
limit_conn_zone $remote_user zone=parora_user:1m;      # per account
limit_req_zone $remote_user zone=parora_rate:1m rate=30r/m;

# server {} context (the meharry.edu HTTPS server block)

location /academics/schools/school-of-applied-computational-sciences/parora/
{
    auth_basic "PARORA - authorized users";
    auth_basic_user_file /etc/nginx/parora.htpasswd;
    # allow 10.0.0.0/8; deny all;                # optional campus-only

    limit_req zone=parora_rate burst=20 nodelay;
    client_max_body_size 50m;
    include /etc/nginx/snippets/parora-proxy.conf; # the proxy_* lines from
6.1
}

location /academics/schools/school-of-applied-computational-
sciences/parora/_stcore/stream {
    auth_basic "PARORA - authorized users";
    auth_basic_user_file /etc/nginx/parora.htpasswd;

    limit_conn parora_total 10;      # max simultaneous sessions site-wide
    limit_conn parora_user 2;        # max tabs per person
    limit_conn_status 503;
    include /etc/nginx/snippets/parora-proxy.conf;
}

error_page 503 /parora-busy.html;
location = /parora-busy.html { root /var/www/parora; internal; }
```

Put all the `proxy_*` lines and `proxy_read_timeout / proxy_buffering` from 6.1 in `/etc/nginx/snippets/parora-proxy.conf`. Create `/var/www/parora/parora-busy.html` with a short message such as "PARORA is at capacity. Please try again in a few

minutes."

When the cap is reached, the next person sees the page but it cannot connect. Streamlit shows its own "connection error" banner and retries, so the busy page matters mainly for direct loads.

### 7.3 Matching the model to the cap

- `OLLAMA_NUM_PARALLEL` (step 5.1.4) is how many questions the model answers at the same time. Others wait in a queue. Most users spend most of their time reading and rotating the 3D view, so 10 open sessions with `NUM_PARALLEL=2` is reasonable on one GPU.
- If users report long waits, lower `parora_total` or raise `NUM_PARALLEL` (needs more GPU memory).
- Watch load with `nvidia-smi` and `ollama ps`, and count live sessions with `ss -tn state established '( sport = :8501 )' | wc -l`.

### 7.4 Account lifecycle

Keep a simple list of approved people, their username, and an end date. Remove accounts at the end of each term with `htpasswd -D`. Passwords go to users individually, never by group email.

## 8. Running it day to day

| Task                | Docker (Option A)                                             | conda (Option B)                                |
|---------------------|---------------------------------------------------------------|-------------------------------------------------|
| Start / stop        | <code>docker start parora / docker stop parora</code>         | <code>sudo systemctl start parora / stop</code> |
| Starts after reboot | Yes ( <code>--restart unless-stopped</code> )                 | Yes ( <code>enable</code> )                     |
| Status              | <code>docker ps</code> , <code>systemctl status ollama</code> | <code>systemctl status parora ollama</code>     |
| App log             | <code>protein-viz-agent/logs/parora.log</code>                | same                                            |
| Service log         | <code>docker logs -f parora</code>                            | <code>journalctl -u parora -f</code>            |

**Updating PARORA** when the project owner releases changes:

```
cd /opt/parora/PARORA && sudo -u parora git pull
# Docker: rebuild and recreate
docker build -t parora -f protein-viz-agent/Dockerfile . && docker rm -f
parora # then rerun the docker run command from 5.2
# conda:
sudo systemctl restart parora # run.sh re-syncs requirements.txt on start
```

Updating restarts the app and ends all open sessions, so schedule it outside class hours. Update Ollama with the same install command; update the model with `ollama pull qwen2.5:7b`.

### Housekeeping:

- Clear old user files monthly, e.g. a cron job: `find /opt/parora/PARORA/protein-viz-agent/{structures,prepared,membranes,simulations} -type f -mtime +30 -delete`.
- Logs rotate automatically inside the app; still include `logs/` in log retention policy.
- Back up only `config.yaml`, `.streamlit/config.toml`, the `systemd/nginx` files and `parora.htpasswd`. Everything else can be rebuilt from Git.

### Security checklist:

- Ports 8501 and 11434 are bound to localhost or firewalled (`sudo ufw default deny incoming; sudo ufw allow 443; sudo ufw allow 80`).
- App runs as the unprivileged `parora` user, never root (the Docker container runs as root inside; host networking plus the localhost bind keeps it off the network).
- Uploads limited to 50 MB and to `.pdb`, `.ent`, `.cif` files by the app.
- Outbound HTTPS allowed to: `files.rcsb.org`, `data.rcsb.org`, `rest.uniprot.org`, `alphafold.ebi.ac.uk`, `www.ebi.ac.uk`, `opm-back.cc.lehigh.edu`, `opm-assets.storage.googleapis.com`, `ollama.com` / `registry.ollama.ai` (model downloads), `pypi.org` and `github.com` (updates). Users' browsers also load the viewer from `unpkg.com`.
- `search.foldseek.com` is only contacted if a user explicitly agrees to an online similarity search, which uploads that structure. Block it at the firewall if uploads to third parties are not acceptable.
- Users should not upload confidential or unpublished structures: uploaded files are stored in a folder shared by all users.

## 9. Go-live checklist and troubleshooting

- ☐ Server meets Section 3; `nvidia-smi` works (GPU servers)
- ☐ Ollama running, `ollama list` shows `qwen2.5:7b`, `OLLAMA_NUM_PARALLEL` set
- ☐ PARORA running as `parora` user, starts on boot
- ☐ `.streamlit/config.toml` has the correct `baseUrlPath` and `gatherUsageStats = false`
- ☐ Local test `curl -I http://127.0.0.1:8501/academics/.../parora/` returns 200
- ☐ Proxy block added, WebSocket headers present, HTTPS works
- ☐ Basic Auth or SSO on, approved accounts created
- ☐ `limit_conn` caps set; busy page in place
- ☐ Ports 8501 and 11434 not reachable from another machine (`nc -zv <server> 8501` fails)
- ☐ Cleanup cron job added
- ☐ Test end to end from off campus: log in, type "show hemoglobin and color by chain", see the 3D model

| Symptom                                            | Likely cause                                           | Fix                                                                                                                                                   |
|----------------------------------------------------|--------------------------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------|
| Page loads, then spins or shows "Connection error" | WebSocket not proxied, or user cap reached             | Check <code>Upgrade / Connection</code> headers and <code>proxy_http_version 1.1</code> ; check nginx error log for <code>limiting connections</code> |
| Blank page or 404 on scripts                       | <code>baseUrlPath</code> does not match the proxy path | Make both exactly <code>academics/schools/school-of-applied-computational-sciences/parora</code>                                                      |
| "Cannot reach Ollama" / no answer to prompts       | Ollama down or wrong <code>OLLAMA_HOST</code>          | <code>systemctl status ollama</code> ; <code>curl http://127.0.0.1:11434</code> ; in Docker, confirm <code>--network host</code>                      |
| Answers take over a minute                         | CPU-only inference or too many parallel users          | Add a GPU, lower <code>parora_total</code> , check <code>ollama ps</code> shows GPU use                                                               |

| Symptom                                           | Likely cause                               | Fix                                                                             |
|---------------------------------------------------|--------------------------------------------|---------------------------------------------------------------------------------|
| "model not found"                                 | Model not pulled for the service's user    | <code>ollama pull qwen2.5:7b</code>                                             |
| Structure fails to load                           | Outbound HTTPS to RCSB/UniProt blocked     | Allow the hosts in Section 8                                                    |
| Simulation/membrane/image tools say "unavailable" | Add-ons not installed (expected on Docker) | Use Option B with <code>setup_tools.sh</code> if needed                         |
| Upload rejected                                   | File over 50 MB or wrong type              | Raise <code>maxUploadSize</code> and <code>client_max_body_size</code> together |

For PARORA behavior questions, contact the project owner, [@Methun](#). Include the relevant lines from `protein-viz-agent/logs/parora.log`.